

Представляем нашу серию F. УЗНАТЬ БОЛЬШЕ >

ИССЛЕДОВАНИЕ

30 ОКТЯБРЯ 2025 ГОДА

Lumina: создание системы самосовершенствующейся оценки на основе обратной связи с заказчиком

Lumina: адаптивная система оценки, которая учится выносить суждения, как эксперт в предметной области.



Чарльз О'Нил (Базеттен)



Гарри Партридж (Бастетен)

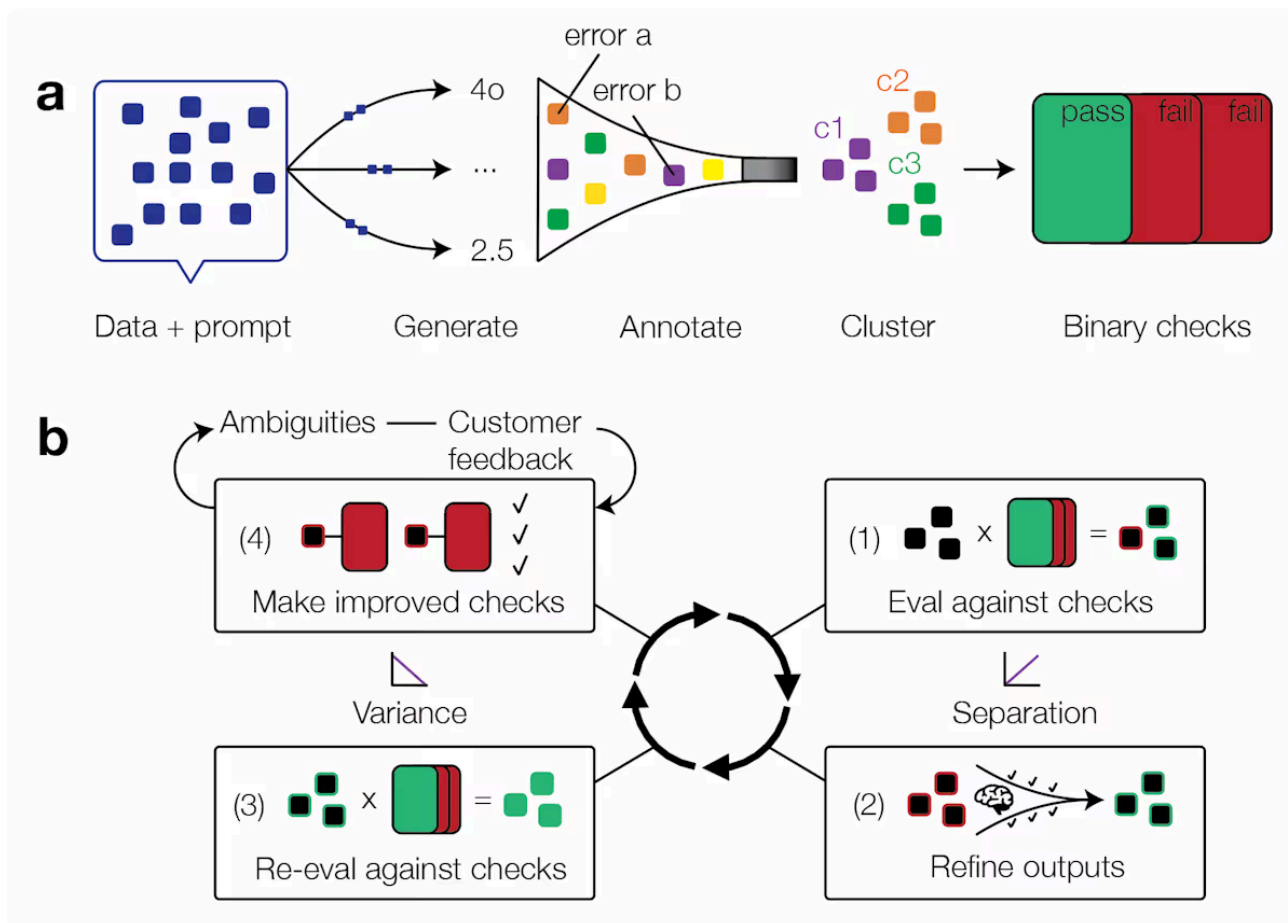


Макс Кинкби (Baseten)



Лжонатон Лю (Бастетен)





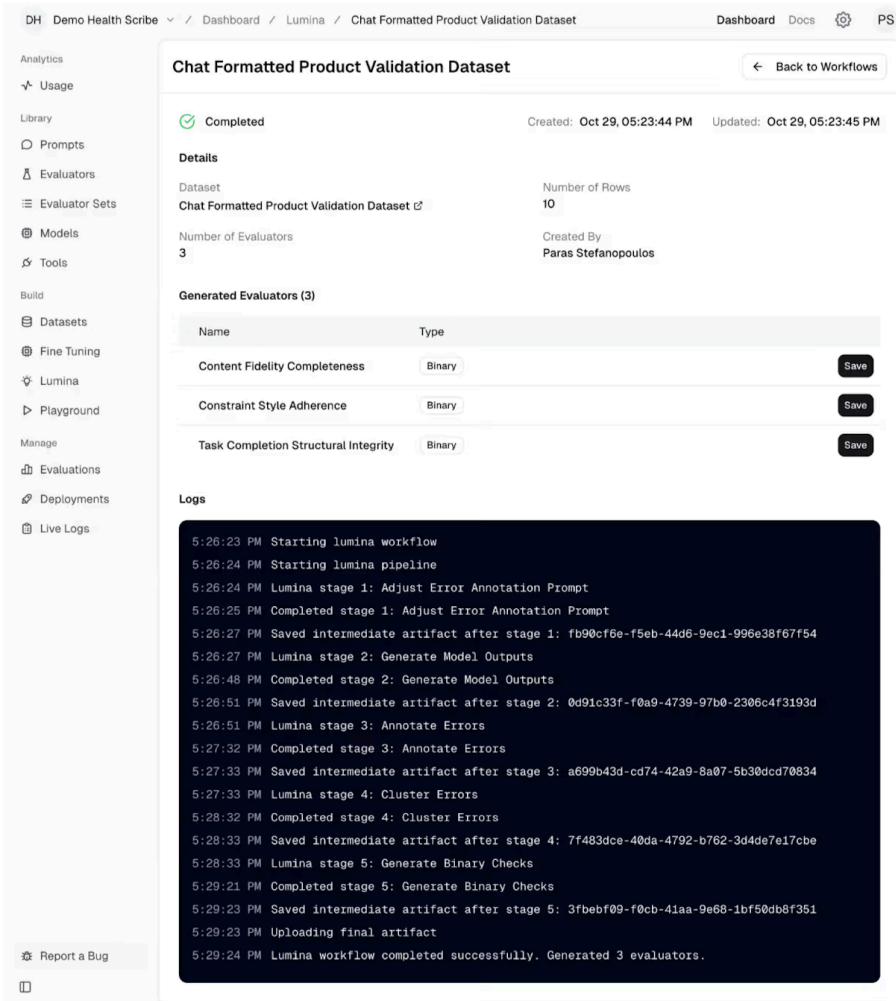
Конвейер оценки Lumina. (a) Этап исследования: на основе данных о клиентах и подсказок генерируются результаты работы нескольких моделей с разным качеством, которые помечаются на предмет ошибок, кластеризуются по семантическим категориям (c1-c3) и преобразуются в бинарные проверки на соответствие требованиям. (б) Итеративный цикл уточнения: оценщики постоянно совершенствуются в четыре этапа: оценка результатов на соответствие требованиям, использование генетической оптимизации для уточнения результатов с целью лучшего разделения, повторная оценка для измерения снижения дисперсии и учет отзывов клиентов о неоднозначных результатах для создания улучшенных проверок. Этот цикл повторяется до тех пор, пока оценки не совпадут с мнением экспертов.

Для нас аксиома использования больших языковых моделей в качестве судей заключается в том, что бесплатного сыра не бывает. Чтобы оценить как выполнение той или иной задачи большой языковой моделью, необходимо привнести в нее информацию из реального мира. Как правило, это выглядит так: мы извлекаем весь скрытый контекст и знания о предметной области, которыми обладает клиент, и объективно, в ясной и систематизированной форме, описываем их в наборе оценочных критериев (большие языковые модели в качестве судей). Вам нужно знать, что именно клиент хотел бы измерить (об этом мы уже писали ранее), а затем точно указать, как это измерить.

В то же время статичные, шаблонные оценки скрывают истинные причины неудач. Теперь мы знаем, что командам нужна система, структура, с помощью которой они

могли бы систематически согласовывать свои оценки с экспертами в предметной области, у которых есть свое мнение о том, что считать «хорошим» и как это определять. Важно, чтобы это происходило динамически, на основе реальных данных, поступающих в эти системы.

Чтобы решить эту проблему, мы создали Lumina. При этом мы начали рассматривать оценку как адаптивный процесс. Он выявляет закономерности в ошибках, совершенствует собственные тесты, уточняет результаты и выявляет неоднозначности, которые может устранить только один настоящий эксперт — заказчик. Lumina — это система оценки, с помощью которой мы можем точно и объективно разделять хорошие и плохие результаты работы моделей. Мы используем данные о клиентах и итеративно «приближаемся к цели» вместе с ними, пока у нас не появится набор моделей, которые будут оценивать выполнение задач.



Lumina работает на нашей платформе. Загрузите набор входных данных для своей большой языковой модели, выберите количество оценщиков и дайте ей поработать.

Lumina — это действительно круто, потому что она дает нам сверхнадежный сигнал для «восхождения к вершине». Именно это позволяет нам начать процесс создания модели с открытым исходным кодом, которая превзойдет по эффективности закрытые универсальные модели, с помощью обучения с подкреплением (используя

набор больших языковых моделей в качестве моделей вознаграждения), SFT (используя эвалюаторы для фильтрации качественных данных для обучения по принципу выборки с возвратом) или наших собственных (гораздо более эффективных) методов, таких как iSFT и RGT.

Создание чего-то, что можно дорабатывать

Lumina начинает работу с общего запроса на аннотацию ошибок (своего рода метазапроса), после чего мы запускаем ряд моделей на таком большом количестве входных данных (часто это десятки тысяч примеров), насколько это возможно с точки зрения безопасности. Мы используем как базовые модели, так и самые современные модели логического вывода, сохраняя сгенерированные результаты. Это позволяет нам метаоценивать широкий спектр «качественных» показателей.

После этого мы позволяем данным формировать саму оценку, используя модель аннотирования для выявления ошибок в сгенерированных результатах. По сути, мы даем очень хорошей модели (обычно Opus) задание проанализировать инструкции по генерации, любой дополнительный контекст, предоставленный заказчиком, и сами результаты и ответить на вопросы: в чем ошибка? В чем ее причина? Что не так с этим конкретным результатом?

We then cluster those errors hierarchically into a taxonomy that spans all observed failures. Rather than acting as labels, these clusters are concrete and executable checks. Lumina turns each cluster into a binary check, allowing an LLM-as-judge to evaluate an extremely precise question and record a structured PASS or FAIL with clear rationale.

We term this process ‘discovery’, providing artifacts for downstream iteration:

1. Take generic error-annotation prompt.
2. Generate outputs with all models.
3. Annotate outputs.
4. Cluster.
5. Create binary evaluation criteria from observed failures.

What do the evaluators (and evaluator sets) actually look like?

Each evaluator examines a single dimension of quality through a specific binary question—for instance, “does this medical note use correct tooth notation throughout?” or “are all

drug dosages within safe prescribing limits?” Rather than asking for ambiguous 1-10 scores where LLMs struggle with calibration, we provide detailed context on how to objectively make each binary decision. The evaluator outputs both a structured **rationale** explaining its reasoning and a PASS/FAIL **score**.

The power comes from aggregation: a set of evaluators targeting different failure modes collectively forms a comprehensive assessment. Their binary outputs sum to create a quasi-Likert score providing both granular feedback (which checks failed with rationales) and an overall metric (18/25 passed). This modular architecture lets us add, remove, or refine individual evaluators without rebuilding the entire framework; each check evolves independently while contributing to the holistic assessment.

Importantly, this is just a starting point. This partitions the error space into semantically related checks that, taken together, should holistically assess the quality of an individual output. However, they're nowhere near perfect yet. Again, you need to inject information, and this has to come from somewhere external, the real world, touching grass.

Evaluation is nothing without its iteration: customer-in-the-loop refinement

These checks seed an iterative loop: we evaluate, refine, evaluate again, and use failures to improve the evaluator itself. In doing so, we aim to ensure that (1) a given check makes the same decision when asked again (low variance), (2) that it cleanly distinguishes acceptable from unacceptable behaviour (strong separation), but most importantly, it (3) **agrees with the customer**.

To extract the customer's latent knowledge about the task, Lumina discovers “ambiguities”, or edge cases, that directly generate questions to be asked to the customer. For instance, if a prompt says “list only confirmed diseases, make no inference” should a listing of “basal insulin and glucose-monitoring supplies” allow the model to infer that the patient may have diabetes? Only a customer can properly provide a true evaluation of these edge cases. We're often surprised by how many implicit thresholds and subjective decision points are inherent in a task. We automatically send batches of these ambiguities, and their questions are automatically fed back into the pipeline to continue to refine the evaluations.

But what about the invisible gaps that might exist between evaluation criteria and the context of the task? Here we use a genetic optimization process to, at each stage (version controlled), generate the optimal output (for every input) with the current evaluation framework. As the customer annotates how closely this output matches ‘perfection’, we are able to continually steer the evaluation criteria toward optimality. If an example is perfect according to the current evaluators, but a customer can spot problems with it, that's a really good, explicit signal we can use to update the evaluators.

Eventually, we converge to a set of evaluators which represent fairly faithfully how an expert from the company would evaluate a given output for a task.

Measuring the quality of evaluations

During iteration, to reduce variance, we measure repeat-judge consistency by re-scoring the exact same input/output pairs multiple times and, where useful, across different evaluator models. We aggregate the results of replicate checks across runs, tracking whether prompt revisions truly reduce variance over time or rather just reshuffle errors.

For separation, we examine the distribution of judgments for “good” versus “bad” examples, monitoring whether these targeted refinements (and thereby prompt versions) increase the fraction of clear PASS/FAIL decisions. In a similar vein, we also examine the consistency of FAIL rationales within a check to understand whether they are pointing at the same underlying criterion. Both of the above signals are continuously fed back into the pipeline to adjust the evaluators.

To complement our methods to reduce variance and increase separation, we also embed static analysis. We analyze evaluation prompts for things such as missing constraints relative to the generation task (when looking at evaluators as a whole), and analyze generation prompts for conflicting instructions with individual evaluators, among other things. It is ultimately these findings that are folded into prompt synthesis alongside our failure evidence, allowing our evaluations to gain specificity.

Finally, we measure agreement with the domain-experts from the company through a bunch of different metrics. What’s cool is that we can feed in arbitrary information about agreement/disagreement (the disagreement itself, the annotation from the customer about why, and along what axis, etc) in order to refine the evaluators further.

Our tracking and improvement of these ‘meta signals’ iteratively improves each stage of our criteria.

What does this mean for training?

The Lumina framework generates meaningful and consistent evaluations that can be repeatedly used for benchmarking model performance. But the important thing is that this can be used in training.

For a defined set of inputs, we are able to run genetic optimization (with Lumina as fitness) to generate ‘gold standard’ outputs that comprehensively satisfy all identified evaluation criteria. Using these golden outputs, only made possible with Lumina, we are thus able to SFT an open-source model to achieve (with increasing data quantity) the frontier level performance of models such as Gemini-2.5-Pro or Claude Opus 4.1.

This does not only apply to SFT: logically, we have observed that RLing open-source models with Lumina itself as a reward function can achieve similar performance, although it's often much slower as you're throwing away a lot of explicit supervisory signal by just relying on the score rather than the rationale as well, which tells you precisely what went wrong or right. We've written a lot about how we achieve much more sample efficient hillclimbing on these evals [here](#) and [here](#).

This pipeline of course has consequences for continually improving performance. As more new examples are observed and evaluated, further edge cases, error categories and ambiguities can be found and eliminated. And using LLM-as-judge as the learning signal is just the start (it's our way to get around the cold-start problem); we eventually integrate all sorts of learning signals that are a bit noisier but closer to the end-user's preferences: scalar feedback, edits, acceptance rates, unstructured text feedback, etc.

Conclusion

Lumina discovers failures, converts them into executable checks and iteratively improves itself with customer judgement. In creating Lumina, we have created an evaluation system that yields benchmarks that are stable, separable and most importantly actionable in being able to steer our model and judge training. The result is not just better measurement but objectively better models. Rather than training on often messy, general customer data, Lumina allows us to train on gold-standard outputs defined by the customer themselves. Of course, it's not perfect yet, and it's also not completely automated, but Lumina has allowed Baseten to build ground truths into the models we provide.

Future work

Our immediate roadmap focuses on moving Lumina from batch-based refinement to continuous online learning. Rather than periodic evaluation cycles, we're implementing infrastructure to run Lumina continuously on production data streams, allowing evaluators to adapt in real-time as new failure modes emerge in deployment. This shift (rolling out to customers shortly) fundamentally changes the feedback loop from weeks to hours. Beyond online adaptation, we're exploring how to explicitly optimise evaluators for discriminative power: not just measuring quality accurately, but maximising the separation between models that genuinely understand a task versus those that merely pattern-match. This involves developing evaluation criteria that expose capability gaps invisible to standard benchmarks.

Finally, we're training specialised judge models for each component of the Lumina pipeline (error annotation, ambiguity detection, criteria synthesis) rather than relying on general-purpose models. These purpose-built judges will bring both efficiency gains and improved accuracy, as models trained specifically for meta-evaluation tasks consistently outperform generalist models prompted for the purpose. Together, these directions point toward a

future where evaluation isn't just a measurement tool but an active learning system that continuously improves both itself and the models it judges.

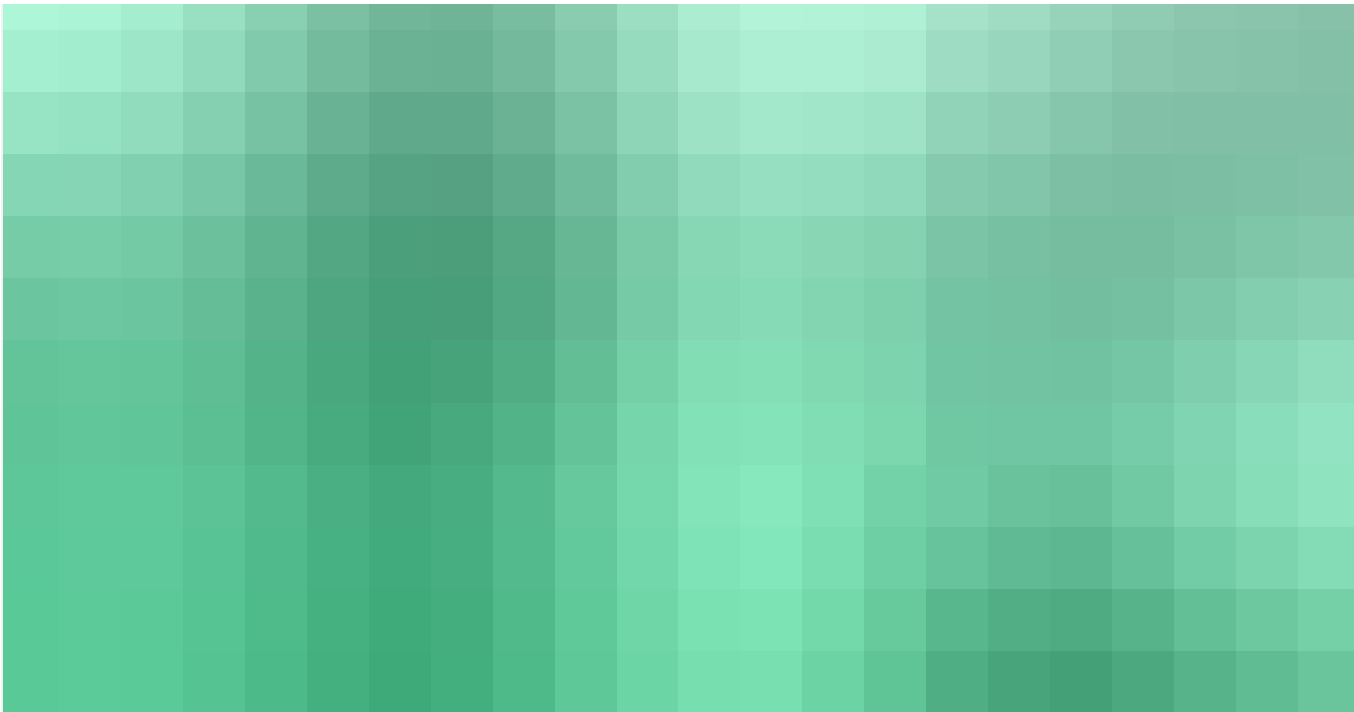
For attribution in academic contexts, please cite this work as:

O'Neill, Charles. "Lumina: Building Self-Improving Evaluation Through Customer-in-the-Loop Refinement." Baseten Research, October 30, 2025.
<https://www.baseten.co/research/lumina-building-self-improving-evaluation-through-customer-in-the-loop-refinement/>

BibTeX citation

```
@misc{oneill2025lumina,  
  author      = {O'Neill, Charles and Partridge, Harry and Kirkby, Max and  
Liu, Jonathon and Stefanopoulos, Paras and Jayasekara, Mudith},  
  title       = {Lumina: Building Self-Improving Evaluation Through Customer-  
in-the-Loop Refinement},  
  year        = {2025},  
  month       = oct,  
  day         = {30},  
  institution = {Baseten Research},  
  url         = {https://www.baseten.co/research/lumina-building-self-  
improving-evaluation-through-customer-in-the-loop-refinement/},  
}
```

Other research



RESEARCH

Post-Training Science for Supervised Fine-Tuning

How the optimal learning rate and batch size move with model scale, family, and data, and whether one selection rule transfers across them.



RESEARCH

Still: Amortized KV Cache Compaction in a Single Forward Pass

You can shrink a language model's KV cache by 200×, in a single forward pass, and it still answers correctly.



CHARLES O'NEILL • 4 others



RESEARCH

Post-training frontier legal agents with Baseten Research

Post-training a 27B open-weight model to the frontier on Harvey LAB with Baseten Research



CHARLES O'NEILL • 5 others

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Product

Dedicated Inference

Model APIs

Training

Frontier Gateway

Baseten Inference Platform

Model Runtimes

Infrastructure

Multi-cloud

Deployment options

Cloud

Self-hosted

Hybrid

Embedded engineering

Forward deployed engineers

Modalities

Transcription

Image Generation

Text-to-speech

Large language models

Compound AI

Embeddings

Industries

Enterprise

Healthcare

Developer

Model library

Documentation

Changelog

Resources

Research

Customers

Blog

Guides

Events

Partners

Savings calculator

Trust Center

About us

Startups

Careers

Contact us

Popular models

- GLM 5.2
- Kimi K2.7 Code
- DeepSeek V4
- GPT OSS 120B
- Whisper Large V3
- NVIDIA Nemotron 3 Ultra
- Explore all

Legal

- Terms and Conditions
- Privacy Policy
- Service Level Agreement

● ALL SYSTEMS NORMAL

